Week 5 — Iteration & Algorithms

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- Loop patterns: counting, accumulating, searching
- List comprehensions
- Linear and binary search
- Debugging loops

Apply — ~30 min

- Code-along: FizzBuzz
- Code-along: number cruncher — min and search
- Code-along: bubble sort walkthrough

Loops as Tools, Not Just Syntax

You already know how to write a loop. This week you'll learn the **patterns** that appear again and again in real code.

- **Counter:** how many items match a condition?
- **Accumulator:** what's the running total?
- **Search:** is this value in the sequence?

Counter and Accumulator Patterns

```
numbers = [4, 7, 2, 9, 1, 8, 3]

total = 0
count_above_5 = 0

for n in numbers:
    total += n
    if n > 5:
        count_above_5 += 1

print(f"Sum: {total}, Above 5: {count_above_5}")
```

List Comprehensions

A compact way to build a new list from an existing one.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

evens = [n for n in numbers if n % 2 == 0]
doubled = [n * 2 for n in numbers]

print(evens)    # [2, 4, 6, 8]
print(doubled)  # [2, 4, 6, 8, 10, 12, 14, 16]
```

Check for Understanding #1

What does this comprehension produce?

```
words = ["cat", "elephant", "dog", "rhinoceros"]
long_words = [w for w in words if len(w) > 4]
```

A) `["cat", "dog"]`

B) `["elephant", "rhinoceros"]`

C) `["elephant"]`

D) `["cat", "elephant", "dog", "rhinoceros"]`

Linear Search

Check each item one by one until you find the target.

```
names = ["Alice", "Bob", "Marcus", "Sara"]
target = "Marcus"
found_index = -1

for i in range(len(names)):
    if names[i] == target:
        found_index = i
        break

print(found_index)    # 2
```

Binary Search — The Concept

Binary search cuts a **sorted** list in half with each step.

1. Check the **middle** element
2. If it's the target → done
3. If target is smaller → search the **left** half
4. If target is larger → search the **right** half
5. Repeat until found or the list is empty

Linear search: up to n steps. Binary search: up to $log_2(n)$ steps.

Debugging Loops

Common bugs and how to spot them.

Bug	Symptom	Fix
Off-by-one	First or last item missed	Check `range()` bounds
Infinite loop	Program never stops	Confirm condition updates each iteration
Wrong accumulator	Incorrect total	Initialize before the loop, not inside it
Overwriting vs. appending	Only last value kept	Use `list.append()` or `+= 1`

Check for Understanding #2

Why does this loop always print `0`?

```
numbers = [5, 3, 8, 1]
for n in numbers:
    total = 0
    total += n
print(total)
```

A) `total` should be a list, not a number

B) `total = 0` resets on every iteration

C) You need `break` after `total += n`

D) `print` is outside the loop

Check for Understanding #3

Binary search requires the list to be _____. Why?

A) Sorted — it uses position to eliminate half the list each step

B) Sorted — it is always faster than linear search

C) Unsorted — it randomizes the search order

D) Short — it only works on lists with fewer than 100 items

Time to Apply

Let's put loop patterns and algorithms into practice.

Mentor 2 takes over

Assignment 5 Overview

Part 1 — Four warmup scripts

- `warmup1.py` — sum 1–100 with a for loop
- `warmup2.py` — input validation with a while loop
- `warmup3.py` — linear search (no `.index()` or `in`)
- `warmup4.py` — FizzBuzz 1–30

Part 2 — Number Cruncher (menu-driven)

- Find min, max, search, bubble sort, quit
- Data file provided in `week-5/data/numbers.py`
- No built-ins: `min()`, `max()`, `sorted()`, `.sort()`

Code-Along 1: FizzBuzz

Print Fizz, Buzz, FizzBuzz, or the number for 1–30.

```
for n in range(1, 31):
    if n % 2 == 0 and n % 5 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

Code-Along 2: Find the Minimum

Loop to find the smallest value — no `min()`.

```
numbers = [42, 14, 78, 5, 61, 29]

minimum = numbers[0]

for n in numbers:
    if n < minimum:
        minimum = n

print(f"Minimum: {minimum}")
```

Code-Along 3: Bubble Sort — One Pass

Understand the core swap before building the full sort.

```
numbers = [5, 3, 8, 1, 9, 2]

for i in range(len(numbers) - 1):
    if numbers[i] > numbers[i + 1]:
        numbers[i], numbers[i + 1] = numbers[i + 1], numbers[i]

print(numbers)
```

Extension Challenge

Ungraded extension: Extend the Number Cruncher with a **Binary Search** option.

Before searching, verify the list is sorted (run bubble sort first if needed). Then implement binary search: check the midpoint, halve the search range, repeat.

Can you make it print how many steps it took compared to linear search?

Wrap-Up

Today you learned:

- Counter, accumulator, and search loop patterns
- List comprehensions for concise filtering and transformation
- How linear and binary search work — and when to use each
- Common loop bugs and how to spot them

This week: Complete warmups 1–4 and the Number Cruncher. Submit via pull request.

Next week: Functions — turning your scripts into reusable, organized code.